



AWS Certified DevOps Engineer - Professional (DOP-C02): Incident and Event Response

Troubleshooting and Rollback

CloudFormation Stack Troubleshooting and Rollback

Hey, Cloud Gurus, welcome! In this lesson, we're going to talk about CloudFormation stack troubleshooting, and if our stack creation fails, how we can successfully roll back. And it's not always an automatic rollback, sometimes we have issues, and we'll need to be able to work around those issues. So in this lesson, we'll talk about some reasons for stack failures. We'll talk about troubleshooting, it's the main focus of the lesson. But most importantly, we'll get into a demo. We'll go to the CloudFormation management console, create a stack, maybe introduce some issues, and fix those issues. And then finally, we'll wrap up with some exam tips. We have the luxury of looking at some issues and solving them without being under huge pressure during a deployment. So let's get started and let's learn about some troubleshooting techniques for CloudFormation. Now, why does stack creation fail? Well, there are any number of reasons that it could fail. Maybe the most common would be insufficient permissions, not being set up properly in IAM. Remember when you're creating a CloudFormation stack, you're touching a lot of different resources if you have a pretty big template and permissions to those other resources are required. Other things that could happen is we could exceed limits. Maybe go beyond the number of EC2 instances that we're allowed to deploy in a region. Remember, that number is not just the number of instances in the stack, it's cumulative throughout your account. There could be other things such as entering invalid parameters. Maybe we entered a security group that doesn't exist in the VPC, and that is just the tip of the iceberg. There's a lot of reasons a stack creation could fail, and we just have to be able to critically think our way through this, troubleshoot, fix our issues, and get our stacks built. All right, let's take a quick look at our diagram. What if stack creation fails? There's a link to the user guide in AWS and certainly some good reading in there. We have a template. What if we're updating a stack? What if that update fails? Worse yet, what if the rollback fails? Why would a rollback fail? Well, it's not the only reason, but do we have drift? Have we run drift detection? So, just think, there are various places where we could have failures. Creation of the template, and also rollbacks, and we'll touch on that momentarily. So, in the upper portion here, we're showing a creation. This is a change set, so obviously, that is an update. And the problem with rollbacks failing is that we could have resources hanging around, not really



doing anything useful, but accumulating charges for us. All right, let's talk about troubleshooting stack failures real quick before we go out and do the demo. Again, there are a vast number of reasons why a stack could fail, and with that, a vast number of troubleshooting techniques, but let's take a look at some first steps. So, when we create a template, especially when we're in the management console creating that template, the main technique is to watch the monitor and the messages we receive during our stack creation, and they're generally very helpful. Now, what about updates? And what if we're having EC2 issues? Well, on the EC2 instances, we can view cloud-init and cfn logs in the `/var/log` folder. We could also install CloudWatch Logs agent on the instance and publish the logs to CloudWatch. And just as common as stack creation failures, we could have rollback failures. Now I mentioned drift a lot, and that certainly could be a reason why a rollback fails. We could have a stack that's rolling back to an old database instance that was deleted outside of CloudFormation. And so we need to check for and remediate drift. And then finally, what if we're deleting a stack and we run into errors? One thing we can do, we have a particular resource that's causing a problem, we can retain that resource, rerun the stack deletion, the stack will delete without that faulty resource, and then we can delete that resource individually. So it's important to understand we can have stack failures during any of these operations. And I've only touched on a few techniques, but each of these have their various techniques. Now, the best way to learn about troubleshooting is just to troubleshoot. And again, this is a luxury. We're not doing a massive deployment to production. This is a lesson, so we can sort of relax, enjoy, and absorb this information. Let's go out to the management console, create a stack. We're going to have some issues, and we'll fix those issues. All right, I'm out at the management console and I have a template ready to go. Let's take a quick peek at this template. I believe this view right here is sufficient. So we're just creating a single instance WordPress application, WebServer security group. We also have an S3 bucket. When creating the template, two reasons, to save time, and also just to keep an element of suspense. I'm not going to say everything that I've done during the creation of this template. So then when we troubleshoot, I'm not giving away all the spoilers, so to speak. So, I always like to validate my templates. And let's just go ahead and create the template. All right, so give our stack a name. Appropriately named. We have to give a few passwords, and I'm just going to use a password of devops. And that's not a security breach, this is going to be deleted, it's not even going to be used. Just give a DBUser name. I'll go t2.micro. And if you're following along, you have to have created an EC2 key pair for it to be available in our drop-down. We're not logging into anything, any instance. So, just create the key pair if you're following along, and it'll be available. All right, some key points here. Notice our stack failure options. Roll back to the last known stable state. We can also preserve successfully created, define a stack policy, rollback configuration. Creation options, termination protection prevents the stack from accidentally being deleted. Just click Next. Here's a review, and we can go ahead and create the stack. Ah, already a problem. This is really good because here's our error message



parameter, DBPassword failed to satisfy constraint: must contain only alphanumeric characters. Passwords I entered were devops for those two passwords, so what's the problem? Well, let's just take a step back here. The problem is, and this only comes from experience because that error message is not descriptive enough, it is actually complaining about the length of this password. Devops is only six characters. This needs to be eight, and this is a constraint of the database. So, make a note of that, maybe not for the exam, but for actually working with CloudFormation. But with that, let's try our stack build again, and let's go ahead and create our stack. And, of course, from the management console, we can view the events of our stack creation, and this will provide valuable troubleshooting material. Remember, though, if you're creating stacks, you're using scripts to create stacks, you don't have access to the management console. There are certain things that we sort of have to know upfront, almost have a checklist for things, such as the makeup of our passwords and the password length. So we have to be aware of those things up front. All right, so our stack created successfully. We got to CREATE_COMPLETE, and we have all of this great information. Now, notice, receive SUCCESS signal. Okay, so, and it's not important which, but what we have is is a wait condition or a creation policy during the creation of our server. And that's important because if we don't get the sufficient number of success signals, maybe multiple success signals in an auto scaling group, guess what, stack creation is going to fail. So there's another thing we want to check obviously with the management console. Information's right here. It's different from the command line, but that information is available as well. All right, so we're good for now. Now let's check on our Resources tab, what was built here, and just like our diagram in the designer WebServer security group, there's our S3 bucket. All right, so let's say several months have gone by and our server has performed beautifully, but it's outlived its usefulness, and we are ready to delete. Remember, six months have gone by, let's say. We don't know everything that's happened on this stack from day to day. Let's go ahead and delete our stack. Ah! Remember, I mentioned that I didn't want to give away everything. What I did was I enabled termination protection on this stack. Now, management console, we get this nice message. If we were operating from the command line, we have to check on this. We have to have this built into our scripts, maybe disabling termination protection. So, our stack's not going to delete because it is protected. All right, let's go ahead and disable it again from the management console. Nice and easy. If we're running scripts, we have to account for this. We have to disable this. We have to have it in our script if it needs disabled. We should be good now, right? Let's try again. Deleting this stack will delete all resources; well, that's what we want to do. Let's go ahead and delete it. Ah, another problem here, DELETE_FAILED. Remember about S3 buckets? The bucket you tried to delete is not empty. Now, remember, I said I didn't want to give away all the spoilers. I went ahead and actually uploaded the template for that stack to this bucket, and that's why our stack deletion failed. Another one of those things, yeah, it's easy and handy from the management console, but we have to realize these things and build these into



our scripts. Let's try one more time to delete. And then we get this, deleting this stack will delete all stack resources. But we have a problem resource here. Stack previously failed to delete because the following resources failed to delete. We can retain resources and deletion of this resource will be skipped. Let's do that. So I'll just go ahead and go back over to Events, DELETE_IN_PROGRESS, but it skipped our problem resource. We can deal with this manually. DELETE_COMPLETE. Our stack is gone. And as you might expect, our problem resource is still here. Here's our stack. There's the contents. So to recap, we created a stack. The stack initially did not build because our parameters were incorrect, our DBPassword was not long enough. Remember that, that error message for that particular case is not very descriptive. Another reason for failure to delete, termination protection. What about S3 buckets? We need to empty those buckets out. Remember when building EC2 instances in a stack, there may be wait policies, creation policies, we have to get those success signals back or stack creation will fail. So that's by no means everything we could run into when troubleshooting, but those are some common ones. And remember, it's a lot easier from the management console. But if we're scripting, we have to be aware for these and we have to account for them in our scripts. All right, let's take a look at some exam tips. Stack failures can occur cause of permission issues. Who's creating the stack? Do they have permissions? If not an admin, do they have sufficient permissions? Maybe we want to create an IAM group, attach the appropriate policies, and put folks who work with our stacks in that group. How about some common CloudFormation stack issues? Limits exceeded. Maybe we're going over our limit of EC2 instances, and that could be a problem. Nested stacks. So, it's reversing the order. When we create, the parent stack is created, the children are created. Deletion, we delete those children stacks first. And again, insufficient IAM permissions can be a problem. My stack won't delete, now what? Try deleting the resource that is causing the problem, or, like we did, we could elect to bypass deletion of that resource. In our case, it was the S3 bucket. Either way, we can then delete the stack. Won't this induce drift? Well, no, because we're deleting everything. Everything is hanging around the stack is gone, so there's nothing to really drift on. Remember, it's not enough to only have CloudFormation permissions; you need sufficient permissions for the resources that are being created as well. That is a look at CloudFormation troubleshooting, issues we can run into creating stacks, updating, deleting stacks, and rolling back. Very interesting. Certainly check out the resources on Amazon, and remember, when you're scripting stack creation and all these stack operations, you have to know this upfront and account for it. And with that, I'm going to end this lesson. Thank you for joining me, Cloud Gurus! I will see you in the next lesson.

CodeDeploy Rollback

Hey, Cloud Gurus, welcome! So in this lesson, we're going to take another look at



CodeDeploy, and we're taking a look from the perspective of this domain incident and event response. We're going to talk about how we can be made aware of problems with our deployments, and also ultimately rolling back, think about auto healing, which is part of this domain. So if we have a problem with our deployment, we want to auto heal. We want to be able to roll back to the last known good deployment. So in the lesson, we'll see how we can utilize SNS to help notify us of events. We'll touch on deployment logs. We've already seen deployment logs, if you recall from the CodeDeploy lessons. We'll talk about stopping a deployment, if issues arise. Then we'll go do a quick demo just to touch on some of the key points related to incident and event response. And we'll check out some exam tips. So with that said, let's take a look at SNS working with CodeDeploy. All right, so how can we monitor and detect issues on our CodeDeploy deployments? We can set up SNS triggers to notify us of issues. So we start doing our deployment with CodeDeploy. Of course, deploying our application to a deployment group. Meanwhile, we've already set up SNS, we've subscribed to an SNS topic, and it's going to monitor our deployment for us. Ultimately, we can be notified if there are issues. And again, just remember all of our triggers and our automations, we could certainly involve Lambda and a Slack channel. So let's talk a little bit about SNS. So we need to set these triggers up pre-deployment. Basically, when you go into CodeDeploy and you're setting your deployment up, these have to be available in the drop-down for you to select them. They have to exist. And we can do this from the console or the CLI. Again, pre-deployment, we set these triggers up on the deployment level, and we'll actually take a look at this in the management console. This is pretty obvious, we must have an SNS topic. Again, when we set up our deployment, they have to exist for us to select them. We can add triggers to an existing application on the instance level. So if we're doing a deployment and we get an email from SNS, hey, there's a problem with our deployment, what do we investigate? We've actually seen this in the CodeDeploy lessons. We'll check out the deployment logs, and if you recall in one of the CodeDeploy lessons, we had an error in the log. They're pretty descriptive, and it's helpful in troubleshooting and sorting out the problem. And we check those out. We can get to those logs through the management console. Again, we saw the error message and solved the problem. And it's going to give us all the information about our deployment, deployment instance, application, the deployment group. Additionally, it's going to give us the error messages, and recall those error messages were in red. So if that's what you're looking for, remember those logs are pretty large. Scan down through and just search for the red text, and that will be the start of your error messages. All right, how can we work with deployments from the CLI? Let's check that out. And I just wanted to show some of the many commands available to us. So we always set our context, `aws deploy`. Make note of that. It's not `aws codedeploy`, it is `aws deploy`, and then our command. And I always remember how literal these are. We want to list the deployments. Well, `list-deployments`, `get-deployment`, `list-deployment-instances`, all of these commands. You certainly don't have to memorize all of these commands. Just make this note, it's `aws`



deploy, not aws codedeploy, and just review these commands. And we can get a great deal of information, as you can see. And I always say anything that we can run from the command line, we can script. So there's certainly some scripting opportunities here. Now, what if we're actually watching a deployment take place? We see something go wrong, can we stop that deployment? And the answer to that is yes, we can. And certainly when we stop the deployment, then we start thinking about, hey, we need to roll back, get back to a last known good deployment, we fix whatever the problem was, and when we're ready, we can redeploy. But it's important to be able to stop. So the sooner you stop, the less you have to fix. We roll back, and we'll talk momentarily about automatic rollbacks, and then we can redeploy. Now, if we're working from the CLI, our command would be that `aws deploy stop-deployment`, and you specify `deployment-id`. And you have to realize that you're stopping in the middle of a deployment, be aware of the instance state. And in most cases, you probably just want to start from scratch, start with a new instance and a new deployment. Now, we've done a lot with CodeDeploy already, so I'm not going to do a full deployment, but let's go out and just start setting up a deployment and look at some of our options related to incident and event response and rollback. All right, so out at the management console in CodeDeploy. So let's just start setting up an application. Again, I'm not going to do the whole thing. Do our application name. I'll select EC2 and create the application. Now, some of the things we can do here is we can set up SNS notifications, CloudWatch alarms for our deployment, and automatic roll back. Now, first thing we can do, ah, notify, we can create a notification rule. Events that trigger notifications. Well, let's just say we want to be made aware of failed events, but we could select all of these if we like. And then we can choose an SNS topic. Now, I'm not going to create an SNS topic. We've done that many times. I'm sure there's one out here. This is fine. And just remember, we have to subscribe to that topic to be notified. So we're creating a deployment, and with that, what we've just done, we'll be notified of any failures when we deploy. So there's our notification rule. Let's go back. Here is our application. Let's start creating a deployment group. I'll just give it a name, assign a service role, tag our instances, and I'll just accept these defaults. But what about alarms and also rollbacks? Advanced options. I can create a trigger. But here is our alarm. Now, you have to create these alarms first, which I haven't done. I just wanted to show where this was at, but we would create an alarm for the instance state. And if it's in a failed state, we want to know about it. What about rollbacks right here? So do we want to roll back when a deployment fails? Absolutely. Roll back when an alarm threshold is met? Sure, we can do both. So we can roll back on either of these, and the rollback is going to roll back to the last good deployment. So that's a look at how we can set up incident and event response for our deployments, as well as automatic rollback. Let's take a look at some exam tips. Use SNS to automatically notify of issues. Absolutely, we can do that. Can we use Lambda? Certainly. Remember that trigger in CodeDeploy? We can absolutely use Lambda. We can use SNS triggers as well. We set those up pre-deployment and then referenced them when we're in CodeDeploy setting up



the deployment. Set triggers at the console level, also at the instance level. That's a bit of a trick question, but the triggers are set at the deployment level and the instance level. And just be aware of our syntax. You would think `aws codedeploy` to preface our commands, but it's actually `aws deploy`. And always prepare for the worst. Clean up failed deployments and rollbacks to the last good state, we can do that with our automatic rollbacks, and then we'll be ready to correct the problem and then re-deploy. So that is a look at CodeDeploy from an incident and event response perspective and automatic rollbacks. And with that, I'm going to end this lesson. Thank you for joining me, Cloud Gurus! I will see you in the next lesson.

Troubleshooting Failed Deployments

Hey, Cloud Gurus, welcome to this lesson, Troubleshooting Failed Deployments. And specifically in this lesson, we're going to talk about troubleshooting CodeDeploy deployments. We'll look at CodeDeploy to EC2, and then some general troubleshooting tips. And we'll have a little demo and look at a tool we can use to help us troubleshoot. And finally, we'll look at some exam tips. So let's go ahead and look at troubleshooting CodeDeploy deployments to EC2. All right. So we have an application in CodeDeploy, and we created a CodeDeploy deployment group, and that deployment group is an EC2 instance, a group of EC2 instance in an auto scaling group, and we run into problems. So what could go wrong here? And really when we get into deploying with CodeDeploy, it's good to have a checklist. So, from that checklist, we can ask some questions. All right, we have lifecycle hooks. Is there an issue with our lifecycle hooks? What about our AppSpec file? Well, first of all, do we have one? We certainly should. Is it formatted properly? There just may be errors or an error in our AppSpec file. Has the CodeDeploy agent been installed on our EC2 instances, or maybe we attempted to install it and something went wrong. Do we have an instance profile? And certainly with CodeDeploy, you need to set up, recall from the CodeDeploy lesson, set up tagging. Now, we wouldn't even get this far if we didn't have tagging set up, but it wouldn't hurt to have tagging in our checklist as well. Let's talk about some general items. All right, a little bit more on troubleshooting. Again, this can also contribute to our checklist, some things we can look at. Do we need a key pair? Was a key pair created? Was the instance tagged? Now, when you create the deployment group, it's going to look for that instance by tag. So you might not get very far without tagging your instance. Check your application and deployment group settings. Setting up the application really is a little bit more simple than the deployment group settings. There are several settings in the deployment group, so you can go to that page in the management console and just step by step take a look at the settings that you have, review that, and make sure everything's okay. Now, specifically, check that the application revision is in the proper location, could be S3 or GitHub or wherever, but obviously, if the path you have entered for that app



revision is incorrect, your deployment is going to fail. So, certainly double-check that. Does CodeDeploy have permission to download the revision? So there's a policy in there somewhere. Now, policy is attached to a service role. Is there a service role? Is it configured properly? And again, back to the policy. Do we have the proper permissions? So, from these two pages, we can certainly create a checklist. Let's go head out to the management console and look at troubleshooting and deployment, and just up front, I'll say this deployment is in pretty bad shape. I intentionally created errors just to help demonstrate the troubleshooting. So let's check that out. All right, out at the management console at CodeDeploy, and let's go to Deployments. So this deployment, just attempted this deployment, I knew it was going to fail because I created errors in it. So this needs troubleshooting. Now, certainly we can go into this deployment and manually start looking at our settings, see what's up with this thing, see where we have mistakes. But what I want to show is another way that can be very helpful if our initial attempts have failed. So let's go over to Systems Manager. All right, in the Navigation pane, Change Management, let's go to Automation. And we want to choose Execute automation. And we're already on Owned by Amazon, that's the one we want. And there's an automation document, `AWSSupport-TroubleshootCodeDeploy`. I want to show this, but certainly want to emphasize if we have our checklist for CodeDeploy, when we have all those things in place, we know how to verify our checklist, that's our first line of defense, know how to use the tool, rather than just kind of rely on this as sort of a crutch, but certainly this can definitely be a great fallback. So let's find that automation. And we want to click the radio button. So we click the radio button on the card, it tells us a little bit here. So we're going to have to provide `InstanceId`, `DeploymentId`. Let's click Next. Simple execution on targets is fine. All right. Now, for the `AutomationAssumeRole`, note it's optional. IAM role which Systems Manager will assume to execute this automation. Now, if it's not specified, Systems Manager automation uses permissions of the IAM user role that runs the runbook. So, long story short, I'm in as an admin, I have these permissions. I don't need to add a role, but we do need `InstanceId` and the `DeploymentId`. Let's go back over to CodeDeploy. Here's our `InstanceId`, simple deployment to one instance, and here is our `DeploymentId`. Some optional CloudWatch alarms and tagging, but let's just execute this. All right, execution has been initiated, and it's going to do `VerifyDeployment`, `CheckInstanceDetails`, and `TroubleshootCodeDeploy`, and certainly this is the step where we're going to get our information. So it's saying success of the automation completing. We still have problems. So, verify deployment. Not telling us a whole lot. `DeploymentId`, here's our instance. `CheckInstanceDetails`. `InstanceState` is running in this subnet, but troubleshooting CodeDeploy. All right, now we're getting somewhere. Deployment failed in `ApplicationStop` lifecycle hook. The instance does not have an instance profile associated with it. So, how do we fix this? Well, we have to attach an instance profile and then restart the CodeDeploy agent. Now again, that would be part of our checklist, and we'd probably figure that out before we got to this



step, but Systems Manager Automation can be a handy tool if we have some troublesome errors regarding a deployment. All right, let's wrap up with some exam tips. Which key components of a deployment should you check when a deployment fails? Well, only three listed here, but key ones, AppSpec file. Have we configured it properly? Are there any errors in it? Is it in the right place? Instance profile. Do we have one? It'll be a problem if we don't, as we just saw. CodeDeploy agent. Does our EC2 instance have the CodeDeploy agent? Now, are we using an AMI, which has that agent on it? Do we need to install it? Either way, we need to have the CodeDeploy agent. What role does tagging play in a deployment? Tag specify which instance, instances also, plural, are included in a CodeDeploy deployment group. Basically telling CodeDeploy, hey, this is where we want to deploy to. So, thank you for joining me in this lesson, Troubleshooting Failed Deployments, with a focus on CodeDeploy. I'll see you in the next lesson!

Event-Driven Actions

Event-Driven Automated Actions

Hey Cloud Gurus, welcome! So in this lesson, we're going to talk about event-driven automated actions, and in the general sense, we've talked about this a lot already. EventBridge events, CloudWatch alarms, Lambda functions, and it's just a matter of knowing, can we automate based on the services we're using? I'll go back to the example. We went as far as getting notifications from AWS health services on scheduled downtime for our instances. So, in most cases, you can use a event-driven actions and be notified of such events. And in this lesson, we're going to talk about EventBridge with CodeDeploy and automating notifications based on what's going on with our deployment. And this is really sort of a part two of the last lesson. So we'll start with an overview, look at EventBridge specific to CodeDeploy. We'll look at some status messages, make a cool analogy here, and a quick demo with EventBridge and how we can link that up with CodeDeploy, and finally, some exam tips. So let's first take a look at EventBridge with Codedeploy. Now in the previous lesson, really the part one, and this is sort of part two of that lesson, we work with CloudWatch alarms, also SNS notification, and rollbacks. What we can also do is link EventBridge with CodeDeploy. Now, I'm always making an emphasis, right now, EventBridge, really the same as CloudWatch events. It's renamed with additional functionality. As time goes on, this naming will be less of a a factor, especially on the exams. But EventBridge can be configured to react to our deployment status. So we're doing a deploy to a deployment group. CloudWatch events is keeping an eye on this. And if situations arise, we can do automations based on any events. One particular thing we could do is we could stream our our instance logs to CloudWatch logs. That's one of many things we can do. So events can invoke one



or more targets when a deployment or instance enters the state you specify in a rule. Remember, our target is in a EventBridge. They can be Lambda function, Kinesis, SQS, SNS, and several others, and that list will probably continue to grow. There's just a definition of what CodeDeploy is, and of course, we know that. Remember the CodeDeploy agent must be installed and configured on those instances to be used in a deployment. So let's talk a little bit about CodeDeploy statuses. And in the example I'm using, these are the statuses available to the API. So, you could retrieve these statuses programmatically. And the reason I chose this one is because it reminds me of ordering a pizza online with maybe one of the chain pizza places, and you get an updated status. So some of the valid values are for our deployment created, queued, in progress, baking. That really reminds me of the online apps for the pizza places. Ready, stopped, failed, and succeeded. Now in the management console, I don't believe there are as many statuses, and it's not important to memorize all of these, but gives you an idea of what's going on during a deployment, and also for rollback, what we may need to clean up. And this analogy with the pizza delivery and baking process sort of hopefully will stick in our minds, or at the very least make us hungry. So let's talk about a couple of points to remember here. The instances we are deploying could have the CloudWatch Logs agent installed. Now remember, if the instance is a part of CodeDeploy, it will have the CodeDeploy agent installed. Don't get these confused. But if we have the CloudWatch Logs agent installed, this is something we could utilize during a deployment, during a failed deployment. We might want to check out these logs during a failed deployment and maybe stream these somewhere for analysis. So again, we could use Lambda to stream these, we could send them to CloudWatch, do a lot of things with these logs, and they will help us in our troubleshooting. Now again, this lesson is sort of a part two of the last lesson, and we've seen enough of CodeDeploy. But let's go out to EventBridge since it's renamed and repackaged, and let's check out that interface a little bit and see how we can use it, link it up with our CodeDeploy deployments, and help us with our deployments, and certainly any remediation we need to take if things go wrong. All right, out at the management console, I'm just going to go to EventBridge, and let's create an EventBridge rule. I'm not going to follow the whole way through with this, just give us an idea how we can connect this up with CodeDeploy. Give our rule a name. And we want an event pattern rather than a rule that runs on a schedule. Let's click Next. And event source, we want the default here. We want events from AWS services. Sample event, this is optional can reference a sample event when you write the event pattern. All right, Creation method, Use pattern form, template provided by EventBridge. We'll stick with that. Source, we want AWS services, but note the other option. And the service being CodeDeploy, but just look at the options we have in this drop-down. Certainly don't memorize that, but get an idea of the flexibility here. Again, we want CodeDeploy. All right, event type. And this is where we really get into the details. All events. Any event having to do with CodeDeploy, API Call via CloudTrail, and of course, exam tip, anytime you



see API, you certainly are at the very least considering that CloudTrail is involved. Let's do a state change. Notice our event pattern in JSON. You might have to interpret such code on the exam, just like a multiple choice. All right, event type specification. Well, if we break this down even further to specific types, code deployment, CodeDeploy Deployment State-change Notification, instance state-change notification, let's go with the Deployment State-change. And as we did that, our event pattern changed a little bit. Let's get a little bit more info. We'll go with specific states. Here we go. So as we run CodeDeploy, as we're doing a deploy, these are the states it's going through, and we can choose as many of these state changes as we want. Again, notice our event pattern changed. A typical problem you might see on the exam is something of this nature. Hey, you're setting up an event pattern for CodeDeploy and you want to be notified of start, stop, START, FAILURE, and SUCCESS, and you'd have a four multiple choice code options, and once you know what you're looking for, it's actually fairly easy. All right, what about applications? Now, to be clear, these are CodeDeploy applications. It's not like a web application, it's a CodeDeploy application. Just for a little bit more info, let's choose specific, and then here are all of the CodeDeploy applications. If we went over to the CodeDeploy console, these would be my applications. So I'll just select an app application I've used recently. And now we get more deployment groups. And again, these are CodeDeploy deployment groups, and it matches up with its application. Then we can click Next. Targets. Select an EventBridge bus, API, or another AWS service as a target, defaults to AWS service. Think about this, typically, would we want to invoke? Well, I mean, we have a lot of things here, but emphasize that invoke, we could invoke or trigger a Lambda function. And here are the Lambda functions, if we went over to the Lambda console. And I'll just choose one of these functions. We're not going to go the whole way through this. The key point I want to emphasize was the setting up of EventBridge with CodeDeploy. So I'll click Next. We can add optional tags, review and create. Here is our event pattern again. Again, it's just JSON code. We've seen JSON code in CloudFormation. And when tested on it, you just have to be careful what they're asking for and make sure that your code matches up with what they're asking for. You don't even need the right code. You'll just get a multiple choice if you do get something like this. And then we would go ahead and create our rule. All right, but I'm just going to leave it at that. I wanted to focus on setting this up, integrating an EventBridge rule with CodeDeploy. So in this lesson, we got specific with EventBridge, how we can use it with CodeDeploy to automate our incident and event response. Just remember that's a narrow focus. We can do a lot of things. I'll go back to the example of linking to AWS health service to notify us of scheduled events with our EC2 instances. So let's look at some exam tips, EventBridge can be used to invoke one or more targets. Remember, those targets in the drop-down, we can invoke more than one of those when a deployment enters a specific state. So we just went through setting up a rule, many things we can do there as we saw. Potential targets could include Lambda, Kinesis, SNS, SQS, CloudWatch alarms, and several more. The behavior



of the automation can be drastically different depending on the state of the deployment. So obviously, when we're in different states, if we're in a failed state, it's certainly going to be a different action than being in a success state. And a failed deployment could involve a notification, cleanup, and rollback, and we saw a lot of that in the last lesson, and we just tacked EventBridge events on to what we've already learned. So that's a look at incident event response and using an example of EventBridge with CodeDeploy. And with that, I'm going to end this lesson. Thanks for joining me, Cloud Gurus! I will see you in the next lesson.

Kinesis and Event-Driven Automation

Hey, Cloud Gurus, welcome! In this lesson, we're going to talk about Kinesis and event-driven automation. So there's an increasing emphasis on being able to make real-time decisions based on events. I intentionally used real time because it really focuses on Kinesis when we're talking about real time. And we look at websites, and a customer places an order or bank deposit, it's an event that triggers a next step. Now in AWS, when we think of event-driven architectures, we automatically think of Lambda and CloudWatch, which is now EventBridge events. And that's perfectly fine, but we should also consider Kinesis, depending on the use case. The lesson will certainly consider Kinesis for event-driven automation, take an overview look, we'll look at some specifics on event-driven, and we'll actually look at it two different ways from two different angles, and then we'll talk about some exam tips. So let's take a look at Kinesis and event-driven automation. So the domain we're currently studying in this section is incident and event response. We've talked a lot about incidents. What about regular events and event-driven architectures? Again, EventBridge and Lambdas stand out, and the other services that come to mind, well, given that this is a Kinesis lesson, certainly Kinesis can form the basis for an event-driven architecture. Let's take a closer look. Think of Kinesis. It's certainly on the exam. When you see real time, you want to be considering Kinesis as part of the solution. But to extend upon that term, real time, Kinesis deals real time big data. So Kinesis helps us move that big data around whether it's streaming data or streaming video, moves it around, helps us analyze that data in real time, and ultimately get that data to our data stores. So we have our producers, click streams, IoT devices, metrics, and log files. And in most cases, this is a lot of data. Think of click streams, how much data could be generated? We could have thousands of IoT devices creating large amounts of data for us. We have to manage that data, we have to stream it, analyze it, get it to our data repositories. We can certainly use Kinesis streams to capture this data, and notice there's no Lambda or EventBridge involved. Kinesis can form the basis for our event-driven architecture. What if we need to perform real-time analytics on that data? So maybe we have a customer at the other end, and we're analyzing the information they input and providing immediate feedback to



them. Then we can use Kinesis Data Analytics. And what if this is a very large amount of data, which, in addition to analyzing it in real time, we want to get it to our backend storage options, S3 buckets, Redshift, RDS, Redshift for more detailed analytics. That's where Data Firehose comes into place. It's getting these large amount of data, so think of the expression drinking through the fire hose, large amounts of, in that case, water, but in this case, data. We're getting it to those backend data stores. Looking at it here is an event-driven architecture, and notice key omissions. There's no Lambda function. Kinesis can be used to build an event-driven architecture on its own. And ultimately, we need to be able to handle events and transactions in real time. There's a key term for Kinesis. You hear real time and start thinking about Kinesis. And notice how, again the Kinesis offerings are chained together and reliant upon upstream events. But Kinesis can handle this, and you can build an event-driven architecture with Kinesis. Now let's look at this in a slightly different way. Sometimes just taking another approach really commits the learning of a concept to our brains. So, on one end, we have producers, and we have consumers on the other end. Our producers, IoT, click streams, metrics, logs, bank transactions. Consumers, applications, or maybe data storage, such as Redshift. We need to get this data into Redshift and then we're going to perform complex analysis on this data. And this way it just becomes an issue of, all right, here's our problem, what is the solution? What tools do we use? How do we stream data to our consumers? And maybe I should have emphasized, large amounts of data. Kinesis Data Streams. We need to analyze data in real time. Kinesis Data Analytics. Maybe it is security video. Stream it and analyze it in real time. Maybe link it up with Amazon recognition. We use Kinesis Video Streams. And finally, large amounts of data into storage, Kinesis Data Firehose. So then it really becomes simplified. Okay, we have producers, we have consumers, and what do we need to do with this data, and what is the appropriate tool, and just knowing each of these tools and their use cases. So really, two key takeaways from this lesson. You think about events and event-driven, we're usually thinking Lambda and maybe EventBridge events. But you also want to consider Kinesis for specific use cases. Real time, I'm going to tack on another key term, real time big data. So Kinesis is a player in this. Given the proper use case, we can build event-driven architectures with Kinesis. So let's talk about some exam tips, key things to remember when it comes to Kinesis. Again, yes, EventBridge and Lambda, certainly event-driven. Kinesis can also be considered as well real time big data. And really, if you see real time on the exam, you have to consider Kinesis. Upstream events, IoT, bank transaction, and click streams need to be processed in real time. We better start thinking about Kinesis as part of that solution. Real time usually screams Kinesis in a scenario question. So we can build a event-driven architectures with Kinesis. It just becomes a matter of understanding our use cases. All right, so this is certainly review from a previous Kinesis lesson, but knowing this is good stuff on the exam. Data Streams, collect streaming data, usually big data for real-time analytics. Video Streams, pretty much the same thing, except it's for video. Common use cases, video security systems. Data Analytics, analyze data from an input



source. Oftentimes, here's that chaining them together, oftentimes from Kinesis Streams and event-driven architecture. And finally, Data Firehose. Deliver real-time streaming data to S3, Redshift, OpenSearch, and Splunk, usually large amounts of data. Think of that term drinking through the fire hose. So just know these use cases and know that we can create event-driven architectures with Amazon Kinesis. So with that, I will end this lesson. Thank you for joining me, Cloud Gurus! I will see you in the next lesson.

Security Information and Event Management

Hey, Cloud Gurus, welcome! In this lesson, Security Information and Event Management, we'll see how we can use some security services in AWS in ways that we can conveniently aggregate our findings and automate event management of these findings. So, as I mentioned, by this point in the course, we're down to this domain, incident and event management. There's some overlap in these domains. We'll be seeing services we've seen already, but just with a focus on incident and event management. Always, we want to be able to automate our response to these incidents and events. So we'll take an overview look at some of the services we can use, and then I'm going to focus on specific services, GuardDuty, Inspector, we've of course seen these already. And then we'll take a look at Security Hub. As its name implies, Security Hub will allow us to aggregate our incidents and events into one location. And finally, we'll take a look at some exam tips. So let's take a look at security information and event management. Can we automate our response to security events and incidents? And by now, we probably know the answer, absolutely! It becomes a matter of knowing some of the tools that we can use and how to automate using those tools. A lot of these services we've seen already, GuardDuty, threat detection for our accounts and workloads. Macie, data security and data privacy uses machine learning and pattern matching to discover your sensitive data focused on S3. Inspector helps improve the security and compliance of applications deployed on AWS. So, it's also looking at not only the application, but our EC2 instances, housing those applications. Security Hub, cloud security posture management service that performs security best practice checks, aggregates alerts, and that's important for us, that aggregation, and it's going to be from some of these other services, and enables automated remediation. In the coming lessons, we're going to look at AWS Systems Manager, and Systems Manager has some automated remediation as well. And in fact, Security Hub can delegate to Systems Manager for this task. And we have Trusted Advisor. Now, remember, Security Hub mentions security best practice checks. Trusted Advisor gives security recommendation best practices, and much more. And finally, Config. And we can set up compliance and auditing with AWS Config. So let's check out some of these services, and we know most of these services very well. GuardDuty. So, our GuardDuty findings can be sent to an S3 bucket. How do we automate this? And I'm going to show the answer graphically with a Lambda



function. And I'm repeating myself because these automations repeat themselves. A Lambda function can monitor our S3 bucket. We put GuardDuty findings in S3, or more precisely, GuardDuty puts its findings in S3. This triggers our Lambda function. Remember, we can aggregate GuardDuty findings from multiple counts into one S3 bucket. So how do we automate? Probably not a shock, it's a Lambda function that would automate this for us. All right, what about Inspector? Inspector has predefined rules packages to check against EC2 instances. How do we automate our findings? Well, let's check it out graphically. Inspector is evaluating EC2, and it pulls findings from EC2, and this can simply be sent to an SNS bucket, and the appropriate people subscribe to that bucket, and there we have our automation of these findings. SNS. Now, a relatively new player to this is Security Hub. Let's check out Security Hub a little bit. So we're using a bunch of different security services, GuardDuty, Inspector, Macie. Is there a way to consolidate this information into one view? Well, what's a hub? Everything is going into a hub. So we have Inspector, GuardDuty, Macie. And, remember Macie's looking at our S3 buckets. And the answer to the question is, yeah, we can aggregate into Security Hub. I'm not going to show this in a management console. We just need to know what Security Hub can do for us. And what it can do, in this case, we go into Security Hub, and we can enable these services, Inspector, GuardDuty, Macie, and by doing that, the findings can be sent to Security Hub. So this will be the central location for our findings. Now to be clear, Security Hub can also perform evaluations and produce its own findings. And then I mentioned remediation of these findings. I'm going to save the remediation part for the Systems Manager lessons. But ultimately, again, Security Hub can be integrated with Systems Manager, and then Systems Manager can go out and perform remediation of any findings. As an example, maybe findings on an EC2 instance, Systems Manager can't remediate that. So we answered it graphically, but yes, we can consolidate or another word, aggregate our findings in Security Hub. Takeaways of this short lesson. We're aggregating our findings into Security Hub. And also, how do we automate our findings? Well, it's some of the usual suspects, AWS Lambda and Simple Notification Service. So our findings can trigger one way or another our automators, Lambda and SNS. Let's take a look at some exam tips. As always, understand the use cases for these services. I'm talking about Inspector and Macie, Trusted advisor, GuardDuty, and we've been over that enough. Understand our automation, our usual suspects, EventBridge, Lambda, SNS. In certain situations, Kinesis is another. Other things we can do. We can aggregate security findings into a single S3 bucket, and that would remove the need for manual steps and strengthen our security posture. And if we need a consolidated view of our security posture across several different services, that is Security Hub where we can aggregate all of that data. So that's a look at security incident and event management. With that, I will end this lesson. Thank you for joining me, Cloud Gurus! I will see you in the next lesson.



S3 Events Notifications

Hey, Cloud Gurus! Welcome to this lesson, S3 Event Notifications. In this lesson, we'll review the events in S3 that can trigger notifications. We'll talk about the types of notifications and their destinations, and we'll solidify this knowledge in the management console and just see how we can set up S3 event notifications. All right, let's talk about use cases for event notifications. So, of course, we have our S3 bucket. We could have a notification for creation of a new object, removal of an object, restoring objects, RRS reduced redundancy, but realize we're also using this as a trigger. We could send information to an SQS queue or an SNS topic. Still looking at some of our events, S3 Lifecycle expiration event. Transition events. And of course, we can use these events to trigger Lambda functions, intelligent-tiering archival events. What if we tag an object? Believe it or not, that can be a trigger. ACL, access control list, PUT events. And we can also send notifications to EventBridge. So there's a ton of flexibility here and many different ways we can set up triggers for S3 event notifications. Now, let's see what this looks like in the management console. All right, out at the management console in S3. And I'm looking at my buckets, and I'm just going to choose a bucket I've been working with for the course. So, where do we set up S3 event notifications? Well, let's go to the Properties tab, and let's just scroll down. And a little more than halfway down, we see Event notifications. And we can create an event notification, so I'll click one of those buttons. Give our event a name. Notice some optional information here, but could be important, such as we can limit notifications, for example, for specific types of files. I'm just going to leave the defaults. Object creation. So these are the event types. Specify at least one event for which you want to receive notifications. Note, all object create events, or we can get specific object removal events, or more specific, object restore. ACL events, object tagging. There's our RRS. Recalling the slide in our lesson. Replication events. Something maybe went wrong. Our replication exceeded 15 minute threshold. Here are lifecycle events. This really, really brings that slide and the lesson to life. Then we have our destinations. We could run a Lambda function and just choose one of our functions. SQS queue, SNS topic. And this is really straightforward now, so I'm not even going to complete the demo. We've done these notification chains many, many times. Really key point here is just having an understanding of the types of events for which we can set up S3 event notification. All right, let's wrap up looking at S3 event notifications with EventBridge filtering. And this is somewhat cryptic. We want GHI data sent to an SQS queue. Let's see what we're looking at here. So, we have S3 and we have several folders. And we're putting objects into these folders now, and this is just alphabetically ABC, DEF, GHI, but we want objects placed in our GHI folder sent to an SQS queue. Maybe these are high-priority objects. This could be a banking application and these happen to be objects with sensitive information. We want to deal with them as soon as possible. Now, we could have a custom Lambda function created to do this filtering, specifically send these GHI



folder objects on their way ultimately to the SQS queue. And that entails creation of a custom Lambda function. Well, maybe we don't have the capabilities in-house to code this in Node.js or Python. What other options do we have? Well, we do have other options. An EventBridge event filter can be set up to filter our GHI objects to SQS. And so when this event occurs, an object placed into our GHI folder, it triggers EventBridge, and EventBridge can just forward that along to our SQS queue. Seems simple, but it's eliminating the need for that custom Lambda function. We just have to set up the wiring from our event notification to EventBridge, but no custom code needs to be written for this. So that's a look at S3 event notification. Just be aware of the different types of actions that can be used to trigger an S3 event, and certainly be aware of the downstream services that we are adding in this notification chain. And it's the usual suspects, SNS, Lambda, SQS, EventBridge. So with that, I will end this lesson. Thank you for joining me, Cloud Gurus! I will see you in the next lesson.

EventBridge Filtering

Hey, Cloud Gurus, welcome to this lesson, EventBridge Filtering with Event Patterns. So in this lesson, we'll learn how we can be selective about what events we respond to by filtering events using these event patterns. And in the lesson, we'll use an example, a very simple example that clarifies the point. So, in the example, we will want to take action when a PNG file is received and not take action when other types of files, such as JPEG files, are received. So let's see how we can do EventBridge filtering with event patterns. All right, what problem does EventBridge filtering solve? Well, let's say the logic for our events have become complex. We have our event, event bus, event rules, and ultimately our target. So this is a more complex situation. We have several event rules. The management of these rules is becoming a problem. The solution is we can write more complex event patterns that only match events under very specific conditions. Examples being, let's say a field of the event is within a specific numeric range, or if the event comes from a specific IP address. So, in a sense, we could do whitelisting or blacklisting based on IP address. So again, what can we do? Set an event pattern on the rule to filter incoming events to specific consumers. Now, we're not limited to one event pattern. The demo is going to be simple, really to get the point across, but we could have multiple event patterns to set up the filtering that we want. And ultimately, our target receives the appropriate content based on our filtering. All right, real-world example, we have a Lambda function for processing only PNG files. We need to set up a filter. So, simple diagram, we're receiving PNG and JPG files, and certainly this could cover many different file types. Ultimately, we have a Lambda function. It's going to operate on the files that pass through our filter. And in this case, the filter will only allow PNG files to reach the Lambda function. So, it's going to filter out any other files. So let's see how we set this up in EventBridge. All right, in the Management console, EventBridge. First, we want to go to Event buses. All right, next. I want to click Create event bus. Give the



event bus a name, and just scroll down, and click Create. Next, I want to click Send events. Notice Event bus, there's a default, and I have a couple in here, and of course, I want the one I just created. Then we enter an event source, and this isn't terribly important right now, but we do have to enter something. All right, then we enter a detail type, and for event detail, notice key pairs here. All right, so filename and a .png file is what we're looking for. All right, let's scroll down. Notice we have a Review button. Let's click that. And this is the event message that will be sent to our Lambda function. All right, speaking of Lambda, let's go over to Lambda, and we want to create a Lambda function, and let's go with Python, and that's just a personal choice. And let's create our function. All right. Once the function is created, here's our source, and let's simply print out our event and click Deploy. Now let's do a test. Click Invoke. All right, so our function ran fine. We certainly could have just trusted that it was okay, but never hurts to test. Let's go back over to EventBridge. And we want to be at the main EventBridge page, and we want to create a rule. Give our rule a name. All right, event bus, well, we just created one. This pngFilter. We can just click Next. All right, Event source, Other, custom events or events sent from more than one source. Sample event, that's optional. Creation method. We want custom pattern. JSON editor. All right, so we need an event pattern in JSON. And I want to click this drop-down and select Suffix matching. Now, for this to be inserted into our code, we click Insert. That makes sense. Now, remember our key pair. Filename, VALUE. What type of file do we want? Our filter will let this through. Other file types will not go through. Then we can just click Next. Our target, remember, Lambda function, there's our Lambda function. Click Next. we could add tags if we like. I'm just going to click Next. Click Create rule. All right, now, check this out, this is where it gets a little tricky. So, my instinct is to look for my rule, make sure it's here and enabled. Where's it at? Can't find it. I have a ton of rules. What's going on here? Event bus, notice default. Click the drop-down. Ah, pngFilter. That's our event bus. A-ha, there's our rule. Just remember that and don't panic, it's there, and it's enabled. All right, now let's go back to our send event. So everything's connected now. We have our Lambda function. Let's click Send. Sent. And timestamp. Let's go over to Lambda. All right, over in Lambda, in our function, we want to go to Monitor, and let's click View CloudWatch logs. All right. And we see our action here just a couple minutes ago. So basically, we sent an event to our Lambda function, and with the filters in place, our Lambda function is only going to operate on PNG files that are sent to it. All other types of files are filtered out. So that's a look at EventBridge filtering and using our event bus. And with that, I'm going to end this lesson. Thank you for joining me, Cloud Gurus. I will see you in the next lesson!

Supporting Services

GuardDuty Overview



Hey, Cloud Gurus, welcome! So, in this lesson, we're going to take a look at Amazon GuardDuty, and we will start by taking an overview look at GuardDuty, what it can do for us, what it is. And then we will look at some details, and these details are important. They separated some from some of the distractors. We'll see with GuardDuty on the exam, such as Inspector and Macie. And just to drive home some retention on this, we will look at GuardDuty from the management console. It's a pretty cool service. And finally, look at some exam tips. So with that said, let's go ahead and learn about Amazon GuardDuty. All right. Now, from an overview perspective, GuardDuty is going to provide a continuous analysis on log files, and these log files include VPC Flow Logs, CloudTrail logs, and some DNS logs. Now to continue with our theme of automation, we can certainly add an EventBridge event to keep an eye on things with GuardDuty. So, GuardDuty will analyze our logs, and EventBridge can be configured to trigger on any new findings. And as always, we could use a Lambda function for remediation or an SNS topic. What is GuardDuty? What can it do for us? What do we need to know? Threat detection. GuardDuty offers threat detection. It enables you to continuously monitor and protect. Protect what? This is important. Your AWS accounts and workloads. GuardDuty analyzes streams of metadata generated from your account and network activity found in CloudTrail, VPC Flow Logs, and DNS logs, another important point of what GuardDuty does. It uses integrated threat intelligence, and a key point in this, and it could be a key phrase you see in a scenario question, it uses machine learning to identify threats more accurately. So, with GuardDuty, we can monitor all of our AWS accounts without additional security software or additional infrastructure to deploy. Again, it uses machine learning to detect malicious or unauthorized behavior. And again, this is an automatic process. It's continuously monitoring our accounts. We can review findings in the console, and we'll see that in the short demo, or we can use it with event management, EventBridge, Lambda. These can be triggers for Lambda functions. So this may be a new service for some of us, so let's go out to the management console and check out Amazon GuardDuty. I'm out at the management console in GuardDuty. And it's very possible that you see something different. If you've never used GuardDuty before, you probably have the main page and a Get started button. You would click Get started, and then you would enable GuardDuty. So again, you would choose Get started, and then you would choose enable GuardDuty. Now, the default is standalone account, just your account, but you can manage multiple accounts with GuardDuty. So, I've done all of that, and it brings me to this Summary page when I go into GuardDuty. And I don't have much running now, but I have enough running to generate findings. So once GuardDuty is enabled, it's going to start analyzing your account, and it's going to report security findings. And I do have a few findings, and we'll check that out, but another thing I can do is generate some sample findings. So in the Navigation pane, let's click Settings. And notice GuardDuty uses a service role to monitor data sources on your behalf. Delegated administrator, we can export findings using EventBridge and S3. And down here near the bottom, we have sample findings, which help you visualize and analyze



the finding types that GuardDuty generates. So, like me, you may not have an account with a ton of resources running, but this will give you a good idea of what GuardDuty can do for us. So let's click Generate sample findings. And we can go back to Summary. And again, this is listing the findings for my account. But let's click on Findings. Notice a bunch of sample findings here. And notice the timestamp, it's cut off, but just a minute ago. And thankfully, I don't have this much wrong going on in my account, but the sample findings just sort of give you that broad spectrum of everything that could go wrong, and we could be proactive about eliminating these findings from our life. And notice on the top right, there's a summary here, high, medium, low, and of course, we have a severity list here, and we can sort on that. All right, So the sample findings certainly gave us a good idea of what GuardDuty can do for us. When we're done with that, we could click the top check box to highlight them, go over to actions, and archive all of these sample findings, or we could export them and at some point, we could unarchive them. Right back at the Summary page, I just have one resource running. It is an EC2 instance. And I'll click on all findings, and I haven't archived the sample findings yet, but there is one for my account, root credential usage. API list findings was invoked using root credentials, and it is of a low severity. So that's a look at what GuardDuty can do for us in our accounts. And for the exam, really just be able to differentiate GuardDuty from other services, such as Macie and Inspector and maybe AWS Config, Trusted Advisor. All right, exam tips, things to remember about GuardDuty. So, when you think of GuardDuty associated with our log files, such as event logs, CloudTrail event logs, VPC Flow Logs, DNS logs, machine learning is used to identify and prioritize potential threats in our log files. And remember those icons of levels of severity. Some machine learning went into categorizing those threats. Our findings can be delivered to S3, or again, as always, just about used to trigger EventBridge or Lambda. And let's take a look at one more bonus tip, and I'm just going to hammer home this point. It usually comes up on the exam, GuardDuty and it's distractors, Inspector and Macie, especially Inspector. Inspector provides security assessments of our applications and how they're set up and configured, whereas GuardDuty is our overall account. And again, with GuardDuty, think of those logs, VPC Flow Logs, DNS logs, and of course, CloudTrail logs. Macie, when you think of Macie, think of S3. So it's analyzing the data security and privacy of our data in Amazon S3. Again, if you know the difference between these three, it'll probably get you some pretty easy points on the exam. So that's a quick look at Amazon GuardDuty. Thank you for joining me, Cloud Gurus! I'll see you in the next lesson.

AWS Inspector Overview

Hey folks, welcome back! So in this lesson, we're going to take a look at Amazon Inspector. So the coverage of Inspector on the certification exam is such that you just need to know the use cases for Inspector and also make sure you know the



use cases of its common distractors on the exam, such as GuardDuty, Trusted Advisor, and oftentimes Macie as well is thrown in there to confuse you. So here's a key sentence for Inspector, Inspector tests the network accessibility of your EC2 instances and the security state of your applications that run on those instances. If you know that about Inspector, you'll be just fine on the exam. So in this lesson, we'll just take an overview look at Inspector. And just to ingrain those concepts, we'll go out to the management console, do a quick demo, and we'll take a look at some exam tips. So we already answered this question, what is the purpose of AWS Inspector? Inspector performs an automated security assessment and helps improve the security and compliance of applications deployed on AWS where they deployed on our EC2 instance. So again, it's assessing our instances, as well as the applications on those instances. An Inspector assesses the applications for exposure vulnerability and deviations from best practices. Now that best practices, that can sometimes be confused with what Trusted Advisor is doing. But Inspector is really just focused on and only on EC2 and the applications on EC2. So, Inspectors is analyzing our applications? Well, yes, but our applications run on EC2. Inspector analyzes EC2 for vulnerabilities as well. Can we automate with Inspector? You certainly probably know the answer to this. Inspector can interact with a Lambda function both from an input perspective and output, as well as SNS. So, as an example, Inspector, let's say Inspector performs an assessment, it's going to produce a detailed list of security findings, and we could have an event, we could trigger a Lambda function, and as you know, Lambda, we can pretty much do anything we can program, but the Lambda could analyze or send that information for analysis. It could notify a Slack channel, or we can notify SNS. So yes, we can certainly automate. How can Inspector be configured. So it's going to be configured to automatically analyze our instances and applications for vulnerabilities. And we can, again, continue the automation process. Let's look at a diagram. Let's say we deployed an application through CodePipeline. We trigger a Lambda function. Lambda can trigger Inspector. Lambda can perform tasks on EC2. Inspector can analyze EC2 and our applications. And ultimately, I'm showing SNS. We can send SNS notifications. As with many of our other services, we could have another Lambda function triggered by Inspector and ultimately notify a Slack channel. So, Inspector's pretty straightforward. We don't have to go real deep on this. Just remember what it does. Again, it tests the network accessibility of our EC2 instances, and the security state of our applications that run on those instances. It's assessing the applications for exposure vulnerability and deviations from best practices. If you know that, you will have no problem with Inspector on the exam. With that said, let's do a quick demo. Demos are sometimes impactful in just ingraining some of these key concepts. So let's go out to the management console. All right, I'm out at the management console in Amazon Inspector. let's talk about some key points before we get started. So, Inspector is going to scan our EC2 instances, ECR containers, and Lambda functions. Inspector is a regional service, so these findings are just for this region. Now, I've already activated Inspector and already



have done a scan, which will look at some results here momentarily. But when you go into Inspector and it is not activated, you will see a Get started button. You click that, and then you click Activate Amazon Inspector. When you activate Inspector in a standalone account, all scan types are activated by default. So, ECR, EC2, Lambda. And when you activate Inspector, it's going to discover your resources and begin scanning all eligible resources. And in this case, it did scan ECR and Lambda, it just didn't come up with any findings of note. And we'll see as we scroll down, there would be more detail here, but nothing really to detail. Now, here's a key point, notice that it also produces findings on AMIs, and it makes sense, as those are the template for our EC2 instances. All right, so we could get at critical findings here, but let's just work with 2 and this AMI. So, view all instance with findings. Click on the instance ID. All right, 3 critical, 5 high, 79 medium. Let's check out one of these critical findings. A flaw was found in lib. Uninitialized variable is used in function ReadSymbol. Now, just scrolling down here, it gives you a remediation. Upgrade your installed software packages, and even the command. Vulnerability details, let me just click that. We have some unformatted JSON, but we can kind of read through this. Highest threat from this vulnerability is to data confidentiality integrity as well as system availability. All right. And all of our critical findings seem to be of that same flavor. Now, we have this note, Inspector now supports deep inspection of EC2 instance. Let me click Activate deep inspection. This can take a few moments to complete. All right, and what is deep inspection? Inspector detects package vulnerabilities for app programming language packages in Linux-based EC2 instances, just Linux. And it is limited to right now Java, JavaScript, and Python. So what it does is scan the default paths for where we're storing our package libraries, and it will find vulnerabilities again, Java, JavaScript, and Python. So in recap, if we just understand what Inspector does for us, it's scanning our workloads EC2, ECR, Lambda functions for vulnerabilities and also categorizes those vulnerabilities. We will be in good shape with Inspector on the exam. All right, in this lesson, we took an overview look at AWS Inspector. If you take anything away from this lesson, just realize that Inspector is analyzing our EC2 instances and the applications that run on those instances for vulnerabilities. If you know that much about Inspector, you'll be fine on the exam. Let's check out some exam tips. So we know what Inspector does, distinguish it from distractors. Common distractors, Macie and GuardDuty, all related to security, but Inspectors for EC2 and applications. Assesses our applications for vulnerability, analyzes the instances that we run on our applications. I'm being repetitive here, but, again, if you know that, it's easy points on the exam. Detailed list of findings prioritized by severity. We just saw that, and it's really cool. So that's a look at AWS Inspector. With that, I'll end this lesson. Thank you for joining me, Cloud Gurus! I will see you in the next lesson.

AWS Systems Manager OpsCenter



Hey Cloud Gurus! Welcome to this lesson, Systems Manager OpsCenter. In this lesson, we'll answer the question, what is OpsCenter, talk about how it works, and then we'll do a demo, just sort of setting up OpsCenter and integrating it with CloudWatch. And we'll wrap up with some exam tips. So to get started, let's answer the question, what is OpsCenter? OpsCenter is a capability of Systems Manager, and it provides a central location where DevOps engineers can manage operational work items. Well, what's a work item or an OpsItem? It's any operational issue or interruption that needs investigated and remediated. Now, the key to OpsCenter is that you can investigate and remediate all within OpsCenter in one location, one dashboard, so to speak. So let's go a little bit deeper here. So again, it provides a location to view, investigate, and resolve operational issues, otherwise known as OpsItems. So with this central location, we can reduce the meantime to resolution for issues impacting AWS resources, and it can aggregate and standardize OpsItems across services. Now, how do we remediate? Systems Manager Automation runbooks can remediate issues quickly, and also remember that we can integrate OpsCenter with CloudWatch alarms and EventBridge events. So there's that automation. So if we get an alarm, this process goes into action. We can investigate and remediate instantly. All right, let's see a little bit more about how it works. OpsCenter integrations. Again, integration with CloudWatch and EventBridge automatically create an OpsItem in OpsCenter. So again, CloudWatch alarms, EventBridge events. And some common alarm scenarios we can have, database read and write actions reach a threshold, CPU utilization, we could also set up memory utilization, billing thresholds, status check failures for EC2 disk space. What about EventBridge? Ah, it can integrate with Security Hub alerts, DynamoDB, auto scaling, health alerts, and our instance is stopping. A lot we can integrate with. And when we integrate OpsCenter, in the OpsCenter console, it will display information from AWS Config, CloudTrail, and EventBridge. It's going to help us investigate these OpsItems, again, all in that central location, so we can move quickly here. And also remember we can integrate on-premises servers with Systems Manager. OpsCenter is part of Systems Manager, so we can also work with on-premises servers using OpsCenter. All right, OpsCenter is pretty straightforward. Let's just solidify all of this information with a demo. All right, out at the management console, we're actually going to go to CloudWatch first. Let's go to Alarms, and we want to create an alarm. Let's select a metric, and we're going to go with the classic CPU utilization. So we select that metric. So we're watching CPU utilization for this EC2 instance. And whenever CPU utilization is greater than, remember, must be a number 0.6, 60% utilization. Let's go ahead and click Next. All right, now, the default, we see it, it's looking to set up an SNS topic. We want to do more. We're actually going to send it to OpsCenter, and from there, we could remediate the issue. So let's remove that default. Let's add a Systems Manager action. And we are creating an OpsItem. This will create an OpsItem within OpsCenter with a specified severity and category. Ah, 60% utilization, probably a high, maybe not critical. And category performance, let's go ahead and click Next. Then we can name our alarm. Click Next. We can review



what we set up. There's our condition. There's our OpsItem. Let's create our alarm. Just refresh the page. There's our new alarm. All right, now, I don't want this demo to be too long, I just want to give the general idea of OpsCenter. But when our alarm in CloudWatch is triggered, CPU utilization going over 60%, it's going to create an OpsItem. Now, a way to test this, basically, we just need to stress our EC2 instance. One way we can do that is with the fault injection simulator. Stress that, and get the CPU utilization over 60%, and again, it's going to create this OpsItem. So let's check out this OpsItem. There's our CloudWatch alarm. And severity, we set it at high, which translates to 2. It's open. So we have an open item. So let's go a little bit further and let's go into this OpsItem, and we're going to get details on this item. We just have to expand it right there. Here's the key. We know that came from our CloudWatch alarm. CPU utilization went over 60%. And just other specific details, again, the priority is important as well. Critical being 1, high being 2. All right, so we have an OpsItem, we know where it came from, we know what the problem is. Now, from OpsCenter, remember, we should be able to remediate the issue from OpsCenter as well. How do we do that? Well, remember where we're at. OpsCenter is part of Systems Manager, and also scroll down, we have all of these runbooks. Our instance is a t3.micro. So, one remediation action we could take quickly is to resize that instance, make it a larger instance. Now, this is set up just for demo purposes. Obviously, a better option may be to set up auto scaling and maybe trigger our scaling based on CPU utilization or memory utilization. But the demo, that's the remediation action we could take. Let's check out this related resources tab real quick. We can also run our automation from here. Resize is in here somewhere. There we go. We could resize our instance and plug in the desired instance type and then run our automation. One other thing I want to show, remember our integration with these other services, we don't have a config rule set up, but there's some information on our instance all on this OpsCenter page. So again, it's this integration of all this information. Here's CloudTrail as well, some of the events that took place. So it's all integrated. We can investigate and remediate. We can use some of these other services to help us investigate all from AWS Systems Manager OpsCenter. All right, let's wrap up with some exam tips. What can we do with OpsCenter? Well, we certainly just saw that. From OpsCenter, we can view, investigate, and resolve operational work items related to AWS resources, DynamoDB tables, EC2 instances, EBS volumes. Which services integrate with OpsCenter? And this is the key to how it works. CloudWatch alarms and EventBridge events. We can integrate with OpsCenter, and when those events and alarms happen, they show up immediately in OpsCenter. What is the work product produced from this integration? AWS OpsItem or OpsItems. We can actually combine those items. So that is a look at AWS OpsCenter in the Systems Manager. Thank you for joining me, Cloud Gurus! I'll see you in the next lesson.

AWS Firewall Manager



Hey, Cloud Gurus, welcome to this lesson, AWS Firewall Manager. Firewall Manager is a security management service that allows us to centrally configure and manage firewall rules and other protections across the accounts and applications in our organization. And the key term here is centrally configure. We can roll out WAF rules, advanced protections, and several other protections. Let's see how this works. All right, what is Firewall Manager? To simplify your administration and maintenance tasks across multiple accounts and resources for a variety of protections. And let's see a list of these services. Notice, this is not the full list, but certainly the key service is AWS WAF, Shield Advanced, Security Groups, and Network Firewall. Notice Firewall Manager, Network Firewall, not the same service, certainly related. And Route 53 Resolver DNS Firewall. And what are the benefits? So we can protect resources across accounts. Firewall is tightly coupled with AWS organizations. We can protect resources of a particular type, protect resources with specific tags, automatic protection to newly created resources, so when we spin up a new resource, it's going to be under that protection umbrella, a Firewall Manager, if we configure it as such, integration with AWS organizations. Notice above across multiple accounts and resources. And we can create our own rules. So how does this work? Firewall Manager creates findings for resources that are out of compliance for detected attacks, and then sends the findings to Security Hub. With Firewall Manager, you can write one set of rules that replicate across all accounts, and that's that integration with AWS organizations. So our WAF rules and our Shield Advanced and Network Firewall rules can all be configured and managed within Firewall Manager. All right, Firewall Manager with AWS organizations. So we have our organizational units and the applications within each organizational unit. So with Firewall Manager, we can set up our protections again just once and have those rolled out and drill down to protect those applications in each of our OUs. So if you're using AWS organizations, Firewall Manager is a great companion to use for your security posture. So we're setting up protections across our organization, all of our accounts. These protections include, again, WAF rules, Shield Advanced protections, Audit Security Groups, and Deploy AWS Network Firewall. Again, don't get AWS Network Firewall confused with Firewall Manager. When you're using Firewall Manager, think of it as an umbrella under which these other services, WAF, Shield Advanced, Network Firewall operate under that umbrella. All right, understanding protection use cases. And we'll go out to the management console and see a brief demo. A picture's worth a thousand words, just sort of tie it all together. Again, all the services can be used together, WAF, Shield Advance, Security Hub, and certainly Firewall Manager. So we need granular protection of our resources? Use WAF. Worried about the DDoS attacks? Use Shield Advanced. Need a comprehensive view of your security posture? Use Security Hub. Well, we have multiple accounts and multiple organizations. We can use Firewall Manager with WAF and roll our protections out across our organization. So let's see a brief demo out at the management console with Firewall Manager. All right, I'm out at the management console in Firewall Manager. Now, the first thing you will do, not shown here because I've done it



already, is enter your account number, and that sets the account as the administrator account. Recall we're tightly integrated with AWS organizations. Notice also the close relationship Firewall Manager, and on the same page, WAF and Shield. So these are really tightly coupled. Prerequisites, your account must be a member of AWS organizations. So again, when you first start with Firewall Manager, you're setting up your account as the admin account. Account must be the Firewall Manager administrator, must have AWS Config enabled for your accounts and regions, configs tightly coupled with organizations as well. Network Firewall, to use Network Firewall or Route 53 resolver DNS Firewall, the organization's management account must enable Resource Access Manager. So, let's just check this out. Notice, and certainly notice pricing per policy \$100 per month. I don't want to set this up and forget it, but let's check it out. Click Create policy. We can set the region, and what type of policy are we setting up? Are we setting up a policy for WAF, WAF Classic, Shield Advanced, Security group, Network Firewall, or Resolver DNS Firewall. Notice this is a radio button, so we could set these up one at a time. I'll just stick with WAF, but the key point here is, again, these are all under this Firewall Manager umbrella. Click Next. Enter a name and description. And then we start creating these rule groups. Firewall Manager creates this web ACL in all accounts that are within the policy's scope. First rule groups, last rule groups. Define the rule groups to run first and the rule groups to run last when the web ACL inspects a web request. Then in the individual accounts, the account owner can only add rule groups to be run in between the first and last rule groups. All right, and then we can define our web ACL rule a bit, add rule groups. All right, and we've seen this in WAF. I'm going to back out. The point I wanted to emphasize was WAF, Shield Advanced, Security Groups, Network Firewall, Route 53 Resolver DNS firewall, all can be configured and managed with our AWS Firewall Manager, closely coupled with AWS organizations. So we can easily roll this out throughout our organizational units. So with that, I'm going to end this lesson. Thank you for joining me, Cloud Gurus! I will see you in the next lesson.

Amazon Managed Service for Apache Flink

Hey, Cloud Gurus, welcome to this lesson, Amazon Managed Service for Apache Flink. And I wanted to do a quick lesson on Apache Flink because it's sort of a new player on the block, but it has deep roots as a service that we should know pretty well. So let's find out what's going on with Apache Flink, solve this mystery, and get to know this service a little bit. All right, what is Amazon Managed Service for Apache Flink? So, in late 2023, AWS renamed Kinesis Data Analytics to Amazon Managed Service for Apache Flink. So it's not so brand-new, after all. We certainly know a bit about Kinesis and Kinesis Data Analytics. And Apache Flink is a framework and distributed processing engine for stateful computations over unbounded and bounded data streams. And when we think data streams, certainly Kinesis comes to mind very soon, if not first. So we have our streaming



data, and certainly Kinesis Data Stream is a very prominent option, and we stream that to what is now called Apache Flink. Apache Flink and perform computations on that data and send that data further downstream for additional analysis. All right, Apache Flink with SQL. And with this setup, we can run SQL code to perform time series analytics, feed real-time dashboards, and create real-time metrics. All right, on the input side, we have our streaming data, and we want to capture this Data Kinesis, S3, Kinesis Data Firehose, any of this streaming data we can send to Apache Flink. With Flink and with the help of SQL, we can perform our time series analytics, create real-time metrics, feed real-time data to dashboards, or other analytical tools. So, we've certainly seen this. I just wanted to focus on this name change, and it's certainly something that we'll be running into on the DevOps pro exam and other exams in AWS. What are some use cases? Well, again, generate time-series analytics using SQL, feed real-time dashboards, create real-time metrics. And to reemphasize the point, we've seen this workflow many times. Just remember, Kinesis Data Analytics is now Amazon Managed Service for Apache Flink. And as you would expect with this heavy computational focus, Apache Flink can work with machine learning, get our streaming data to Apache Flink. But now we have a machine learning component, and machine learning queries perform complex analysis on data, relying on the history of the data in the stream to find unusual patterns. So, that's very key, we're looking for unusual patterns, and these patterns can help us find anomalies. So, we're running anomaly detection here using two types of anomaly detection, `RANDOM_CUT_FOREST` and `HOTSPOTS`. `RANDOM_CUT_FOREST` takes a random set of data points and builds models from those data points, again, looking for anomalies, whereas `HOTSPOTS` will locate and return information about dense regions in our data with the goal of detecting anomalies. And that's about as deep as we need to go into it. Just recall, Kinesis Data Analytics recently renamed to Amazon Managed Service for Apache Flink. And just understand those use cases. Again, we're getting streaming data, and with Flink, we can perform computations and data analysis, send that downstream to dashboards or for further analysis, and finally, we can also interact with machine learning, and just remember the types of anomaly detection, `RANDOM_CUT_FOREST` and `HOTSPOTS`. So that will end this lesson. Thank you for joining me, Cloud Gurus! I will see you in the next lesson.

AWS ElastiCache Overview

Hey, Cloud Gurus, welcome to this lesson, Amazon ElastiCache Overview. It is a service that seems to turn up a fair amount on the DevOps Pro exam, so I thought a brief refresher lesson would be a good idea. Let's go ahead and dive in on ElastiCache. All right, ElastiCache explained. ElastiCache improves application performance by allowing you to retrieve information from a fast in-memory system. So it's analogous to DAX for DynamoDB, and we're just storing frequently access data in-memory for quicker retrieval, and certainly,



ElastiCache works with EC2. And just remember, again, DAX works with DynamoDB. So we could be frequently retrieving data from RDS and decide let's offload some of that to ElastiCache. The in-memory caching can be used to significantly improve latency and throughput for many read-heavy application workloads. So our app is doing a lot of heavy reading. We can send it over to ElastiCache, take some load off our database, and get data to our end users quicker. All right, let's talk about use cases, and certainly caching. Offload read traffic from the database, but ElastiCache can be helpful for machine learning and AI. It's a fast in-memory data store to help you build and deploy machine learning models quickly. So, certainly remember that, and that might jump up on the exam. What about session management? We can store our session in ElastiCache for online applications, such thing as gaming, e-commerce websites, and social media platforms. We can use ElastiCache for real-time analytics, use cases such as social media, gaming, ad-tech, e-commerce, healthcare, financial services, IoT, and time-series data analytics. So just try to remember these use cases for ElastiCache and certainly how it works. Restoring that frequently accessed data in ElastiCache, our application can get to it and retrieve it quickly. All right, now there are two flavors of ElastiCache in AWS. And the key is understand the use cases in general. Understand when to use Memcached or Redis. They're both popular, open-source, in-memory data stores, and both can be implemented via ElastiCache. So keywords really cuts to the chase with Redis and Memcached, so multi-threaded or multi-threading, you want to think Memcached. Advanced data structures, think Redis. And let's look at a handy checklist, sub-millisecond latency, check for both. Advanced data structures, that's going to be Redis. Multi-threaded, Memcached. Publisher/subscriber, think chat rooms, Redis. Snapshots, Redis. Replication, Redis. And transactions, Redis. So this can be summarized. They're both fast. Sub-millisecond latency, multi-threading, Memcached, just about everything else, most importantly, advanced data structures, think Redis. And we have a relatively new player, ElastiCache serverless. Just be aware of this. It allows you to add a cache in under a minute. You're not dealing with any servers, and instantly scales capacity based on application traffic patterns. Instantly scales. Might be a keyword for ElastiCache serverless, or a key term. And you can do serverless with either Memcached or Redis. And ElastiCache serverless monitors your application's memory, CPU, and network resource utilization, and it will scale instantly to accommodate the changes that your workloads encounter. Let's see how to get started with serverless in the management console. All right, so I'm at ElastiCache in the management console. Let's click Get started, and let's just select Memcached. Notice the default is serverless, use to quickly create a cache that automatically scales to meet traffic demands, no servers to manage. Let's give our cache a name. And since it's new, let's check out default settings. And in version, remember we chose me Memcached. Availability zones is default AZ. Encryption, always enabled. Default security groups, and of course, we can customize this if we like. So, we'd select a VPC, select our AZs. We can customize security settings, specifically encryption and security groups, and set memory



limits for our cache. Just wanted to take a look at that. But with all that said, I'll just use the default settings, and click Create. And I'm timing this. It's a bold statement to expect it to complete in under a minute. All right, and our serverless cache using Memcached has been created. Truth be told, it's probably about a minute and a half. This detailed summary page is actually a good one to go back to and end the lesson on. Just remember, serverless, we can choose either Memcached or Redis. And with regards to Memcached and Redis, if you need advanced data structures, you're certainly thinking Redis, multi-threading, you're thinking Memcached. And with that, I will end this lesson. Thank you for joining me, Cloud Gurus. I'll see you in the next lesson!

Incident and Event Response Review

More Event-Driven Scenarios

Hey, Cloud Gurus, welcome to this lesson, More Event-Driven Scenarios. And as the title says, we're going to talk more about event-driven architecture and the different services that can contribute in event-driven architectures. We'll start off talking about common services, talk a little bit about security automation, and any time we want to kick off events, there's going to be some monitoring involved. We'll talk about S3 events, compliance automation. And when I hear the word compliance, I think about AWS Config right off the bat. And we'll finish up with some exam tips. So let's look at common services and maybe a few that don't immediately come to mind. All right, quick, name three services that are event-driven. Ah, Lambda. That's probably the first one we all think of. CloudWatch, CloudWatch events, and alarms, and, of course, EventBridge events, and there's our EventBridge. Any others come to mind? Well, Kinesis is certainly event-driven. So those are the ones that are probably most commonly thought of when we think about event-driven architectures. Are there even more services that contribute to event-driven architectures? Ah, CloudTrail. Maybe think about some findings in CloudTrail that we want to take action on. We could certainly trigger a Lambda function to help us take action on findings in CloudTrail. AWS Config, maybe we have some compliance issues. And we can certainly use AWS Config to configure remediation actions, and we'll actually take a look at how to set that up in AWS Config. Another one maybe we don't think of so much is S3, but maybe it's just too obvious. If we're using S3 as repository and we have a pipeline, if we drop code into S3, it's going to trigger that pipeline. But we'll talk about some other events that can be triggered in S3. All right, so let's look at security automation. So, picture security automation in your mind. What are we trying to do with it? Typically, we're monitoring something. An example could be maybe an EC2 instance and maybe the ports on that EC2 instance. Are our ports too permissive? We can monitor that and certainly detect that. Security Hub will detect that for us. AWS Config could detect that for us. And finally, we can



remediate, think about the services that can remediate for us. Certainly Lambda. And what about Systems Manager? Maybe we could have a Systems Manager automation remediate a port issue for us. So that's probably the most common scenario, but we can also create. So, based on findings, we could trigger a Lambda function to create security groups or NACLs or policies for us. But wait, there's more! Or we can process. So we get some CloudTrail logs, and we could trigger a Lambda function to either process those logs or move those logs somewhere to where they can be processed. And maybe the overall theme of this lesson is just sort of have an open mind about these event-driven processes and just don't get stuck with maybe the typical two or three that we think of. There are many ways we can do this. All right, so let's talk about security and compliance monitoring. All right, so we have a Lambda function. Ah, Trusted Advisor. Remember, Trusted Advisor can do some security checks for us, but we sort of have to help it along. We need a Lambda function. What about AWS Config? And again, there's that match with compliance. So maybe we have resources that are out of compliance. Certainly Config rules can detect that. That could trigger a Lambda function, and we can download that data from AWS Config. Now, another thing we can do, and we'll show that momentarily, is automatic remediation. But of course, this Lambda function could be doing any number of things for us. And going back to previous screen, Config could detect, hey, we don't even have a security group here. Well, guess what? It could trigger a Lambda function and go ahead and create that security group. All right, let's review S3, and specifically S3 event notifications. So with S3, we're talking about bucket events, and then we have event destinations. Well, certainly, probably the most common would be for an S3 event to trigger a Lambda function. But we could have it tied to an SNS topic. And again, there's our Lambda function. Or SQS, maybe we need to decouple these messages, not deal with them immediately. So we put them in SQS. And of course, we can tie it to EventBridge and set up our automation. Now, this hopefully is just review some common events for S3 event notifications. Let's say a new object is created, and that can trigger an event. We want to know if an object is removed, we can certainly set up an event notification. Maybe to SNS, or another common pattern is Lambda to a Slack channel. Now, this is one that probably would be a bit under the radar, but S3 Lifecycle expiration, we can set up event notifications for that. Or maybe just a transition from an S3 Lifecycle stage from one stage to the next, maybe from regular storage to longer-term storage. Maybe we have intelligent-tiering set up, automatic archival objects, we want to know about that. That could be another event trigger our automations. Object tagging. So, in the lesson, certainly when we think about event-driven architectures, Lambda immediately comes to mind, maybe EventBridge and CloudWatch, but just keep in mind and be aware that other services, such as AWS Config, S3, and CloudTrail can play a part in these event-driven architectures, and just understand how they can. So with that said, let's go out to the management console and go to AWS Config and just see how we set up automatic remediation in AWS Config. All right, out at the management console, and preconfigured some things in IAM,



and with EC2, just in the essence of time, just want to demo this and give an idea of how it works, but I don't want it to be too time-consuming. So let's go into View dashboard, and let's go to on this left-hand navigation, go to Rules, and for the Rules drop-down, let's select Noncompliant, and let's add a rule. And there are a bunch of rules. I'm going to search on instance type, and it gives me desired instance type. Click Next. And we just accept the defaults here, except for let's set a desired instance type as t3.micro. And click Next. And we can review and add the rule. All right, more to do here, but real quick there's go over to EC2. All right, so we have four t3.micro instances and two t2.micros. These are non-compliant. The idea would be to catch these and then do whatever we want to do with the evaluation, maybe send an SNS message, or even terminate the instance, or stop the instance. Terminating seems a bit severe. Let's go back over to AWS Config. So we have this rule, and now what we can do, go to Actions and start managing our remediation. We could set up a manual remediation, and we can do an automatic remediation. Let's set that up. And we're just going to set it up. I think that's sufficient. I don't want to get into a bunch of testing, but we could do AWS stop instance. Now, just think about that. How do we stop instances? Automatically. Well, Systems Manager. So, we have this resource ID parameter, and notice its parameters are down here. Let's select InstanceId. So that's going to pass in InstanceId. It's a actually passing in a like a CSV of however many InstanceIds we have. And then we have to specify this role for SSM. So let's go over to the management console, and I've created this role, AmazonSSMRoleForInstanceQuickSetup. And it has a couple of policies, most notably this SSM Managed Instance Core. So it's allowing Systems Manager to basically do things to EC2 on our behalf. Now, here's where it's just a little tricky. This isn't clear cut. You would think maybe, hey, just plug in the role name. Well, that's not going to work. What we need is the ARN. So we plug that in, and there we go, we've set up our automation. And by the way, it's telling us we have two non-compliant instances. There's our two t2.micros. So that's as far as I'm going to take that. But the key point is, again, we have compliance issues, AWS Config can detect them, but also with the help of Systems Manager, remediate those actions. All right, let's talk about some exam tips. And just as an overall theme, we want to be thinking about event-driven automations, and let's not get stuck in thinking it's just Lambda or it's just EventBridge, and remember those other services that can help us. So what are the destinations for S3 bucket events? SNS, SQS, Lambda, and EventBridge. How can you automate compliance issues? You can set up automatic remediation in AWS Config. You also have the manual remediation option, which I guess in a sense could act as sort of an approval in a pipeline making that analogy. You don't want to automate it fully, you want to have the option to remediate it yourself. What service can you use to set up automatic security checks in Trusted Advisor? AWS Lambda. Lambda can be used to kick off those security checks when you need them. So that's a look at some more event-driven scenarios, and maybe just keeping in mind other services such as S3, AWS Config, CloudTrail, on top of our standards that we think of, such as Lambda and EventBridge. So with that, I will



end this lesson. Thank you for joining me, Cloud Gurus! I'll see you in the next lesson.

DevOps Match Game for Incident and Event Response

Hey, Cloud Gurus, welcome to this lesson, DevOps Match Game for Incident and Event Response. In this match game, there are no cash prizes, but you can cash in by acing the DevOps Pro exam. All right, so this time we just have one game, one very important game, AWS Event Sources. So let's check out this game. All right, AWS Event Sources. We have AWS Health, EventBridge, CloudWatch Events and CloudTrail. And we have terms on both the left and the right. Again, the premise being on the DevOps exam, we have scenario questions, multi-sentences often, and consider these one sentence, maybe one very important sentence where we can pick out a key word or a key term and really speed up our answering process by eliminating incorrect answers. Let's get started. So, with this event source, we can learn how service and resource changes might affect our applications running on AWS. I think this might be a short trip, straight across. That applies to AWS Health. If you see something like this, think about having AWS Health in the correct answer. Provides visibility into resource performance. Tricky one. Let's see where this goes. AWS Health. Now maybe by process of elimination, we got to that answer, but it might point out a weakness in AWS Health. So if you struggled with that, just maybe go to the AWS Health facts and really study up on the key features of AWS Health. Receive, filter, transform, route, and deliver events. EventBridge. With this, we can generate reports on the availability of our AWS services and accounts. Again, heavy on AWS Health on this left side. Probably a knowledge of these other three, again, through process of elimination would have gotten us to AWS Health. All right, this one is no longer the preferred way to manage events. Hopefully this is pretty easy. CloudWatch Events is still around, but AWS recommended way to manage events is EventBridge. Track API events. When I see API, I see that in the scenario question, I'm immediately scanning the answers for CloudTrail. If an answer doesn't have CloudTrail in it based on this bit of information, I eliminate that answer. Don't even bother reading it. Just scan it looking for CloudTrail. This can track an IAM event, such as AttachRolePolicy. There's another one for CloudTrail. Changes made here will also appear in the EventBridge console. So what is so closely linked with EventBridge? Well, I'm thinking of the OG, CloudWatch Events. All right, so those are some quick facts in comparison on our event sources, AWS Health, EventBridge, CloudWatch Events, and CloudTrail. Just knowing the fundamentals of these services, knowing some key terms, such as API, which screams CloudTrail, can speed up our test-taking process and quickly eliminate incorrect answers. So with that, I will end this lesson. Thank you for joining me, Cloud Gurus! I'll see you in the next lesson.



Incident and Event Response Scenarios

Hey, Cloud Gurus, welcome! In this lesson we're going to do our end of section review and take a look at some incident and event response scenarios. So we'll just set up the scenario prep and then we'll just go into the scenarios. Please note that the third scenario on OpsWorks has been removed. OpsWorks is scheduled for end of life very soon and will not be part of the exam going forward. So let's just go ahead and dive into this lesson. Now, if you're taking the course in sequence, you've seen all this already, but just in case you're just jumping into this lesson, we want to read the scenario and try to visualize it first. Visualization is important when we get difficult scenario questions on the exam. After we visualize it, let's try and quickly sketch the scenario on a Notepad, like what we have available on the exam, and then we'll click through and reveal the solution and compare it to your sketch. Most importantly, though, is the visualization part of this. Now, remember, on the exam, we have roughly 2 minutes per question, so we're not going to have time to sketch question on the exam. Oftentimes, if I run into a difficult question, I quickly mark for review and move on from it. I accumulate wins, answer the questions I'm real confident in, and it really does get your confidence up, and you'll find that when you look at a question the second time around, often it's easier. So let's go ahead and look at some scenarios. All right, we need to send a notification if someone has stored access keys on public repositories. So go ahead and pause the video. Think about that. Visualize your solution, sketch it out, and I'm going to go ahead and move forward here. And I'll just say that that's an absolute no-no with the tools that we have to store access keys on public repositories. So, what can we do? All right, let's say we're using a public GitHub repository. There is an EventBridge event, `AWS_RISK_CREDENTIALS_EXPOSED`. Don't be alarmed, we didn't cover this specific event in the course, but just go ahead and look it up and make sure you understand what it does. Ah, we can get a step function involved. We create a CloudWatch events rule or EventBridge for the `AWS_RISK_CREDENTIALS_EXPOSED` event from AWS Health Service. Then we use a step function to delete the keys. Now, I didn't want to get overly elaborate with these scenarios, so that gets rid of the security risk. Now, what we would do, the additional step would be to store these keys in Parameter Store. Now, if we want secret rotation, automatic secret rotation, well, then we can get Secrets Manager into this scenario. So that was kind of a tough one. And honestly, if you thought of another way to do this, that's probably fine, too. Another thing we could do, the event could trigger a Lambda function, we could write to go ahead and deal with this key in a public repository. All right, let's take a look at another scenario. We need to monitor scaling events of our auto scaling group and be notified via email when scaling occurs. Go ahead and pause and think about this, sketch it out, and I'm going to move forward here. So we have our auto scaling group, good old EventBridge event. We're monitoring our group. When a scaling event occurs, triggers our event, which can be attached to an SNS



subscription. Now, the specific thing we're monitoring is EC2 Instance Launch and Instance terminate events. And then we can create a subscription and link it to the events. Whoever subscribes can get an email of these events. And we know these automations by now could be a Lambda function out here communicating with a Slack channel as well. So with that, I'm going to end this lesson. Thank you for joining me, Cloud Gurus! I will see you in the next section.